

**Construction of Solutions for Complex Superfunctions and Iterative Functional**

**Equations**

What are the generalized methods for constructing complex superfunctions and iterative functional equations, and how do these methods contribute to solving and visualizing these functions in the complex domain?

**Pages:23**

## Table of Contents

|                                                 |    |
|-------------------------------------------------|----|
| Table of Contents .....                         | 2  |
| Introduction.....                               | 3  |
| Superfunctions .....                            | 3  |
| Preliminaries .....                             | 5  |
| -Domain Colouring .....                         | 5  |
| -Contour Integrals .....                        | 5  |
| Iterative Functional Equations .....            | 6  |
| -Fixed Points .....                             | 8  |
| -Transforming the Schröder Equation.....        | 13 |
| Construction of the Superfunction .....         | 15 |
| -Critical Points.....                           | 18 |
| Construction of Generalized Abel Function ..... | 20 |
| Extensions .....                                | 22 |
| Conclusions.....                                | 23 |
| Gallery.....                                    | 24 |
| Appendix.....                                   | 26 |
| Bibliography .....                              | 30 |

## Introduction

Functional equations have always been a stump in the area of mathematics, as a general method of computation without extensive trial and error or unique methods for unique questions were required. Functional equations and most importantly superfunctions play an important role in mathematics, but they are generally overlooked or obscured away. Despite their significance in both theoretical and applied mathematics, superfunctions and related concepts, such as fractional iteration and functional iteration, have not received the attention they deserve.

My interest in these functional equations gave rise to googology, an area of mathematics that is dedicated to large numbers primarily for fun; the functions tetration and pentation, and so on evoked my interest. With the knowledge of complex numbers, the will to generalize these fast-growing functions grew on me. With the help of complex numbers solving these equations became possible with Kneser (H. Kneser 1949) although his method of generating tetration was nowhere close to simple and was ungeneralizeable for the most part. With the work of Kouznetsov and progress made in laser science, superfunctions have become more well-known and investigated subjects, as specific functions superfunctions and even super-superfunctions have been calculated and approximated in the complex domain (D. Kouznetsov, 2014). I will elaborate on a more generalized version of Kneser's construction of my own making and explore the visually intriguing graphics. Using Infinite composition and fixed points I will show how to be able to construct such functions.

## Superfunctions

Function composition is defined as  $h=g \circ f$  such that  $h(x)=g(f(x))$ . The functional iteration for a natural number  $n$  is such that  $f^n = f \circ f \circ f \dots n \text{ times } \dots \circ f \circ f$ . and for negative repetition, the inverse function  $f^{-1}$  is used.  $f^{-n} = f^{-1} \circ f^{-1} \dots n \text{ times } \dots f^{-1} \circ f^{-1}$ . The functional root  $f^{0.5}$  is defined as  $f^{0.5}(f^{0.5}(x))=f(x)$  and similarly any fractional iterate can be defined in such a way,  $f^y(f^z(x))=f^{y+z}(x)$ . Superfunctions are diagonalisation of fractional iteration

superfunctions of  $f$ , where  $F$  can be defined as  $F(x)=f^x(z_0)$ . Superfunctions are defined by a functional equation; for a given function  $f$  and an initial starting point  $z_0$  (generally selected to be 1). The solution of the superfunction is the function  $F$  that satisfies these equations. The last equation says that any real input that goes into  $F$  must output another number  $F(x)$  which is also real, which is not the case for other functional equations such as the Abel and Schröder equations. (The overbar used utilized as taking the complex conjugate)

$$F(z + 1) = f(F(z))$$

$$F(0) = z_0$$

$$F(\bar{z}) = \overline{F(z)}$$

$$F:\mathbb{R} \rightarrow \mathbb{R}$$

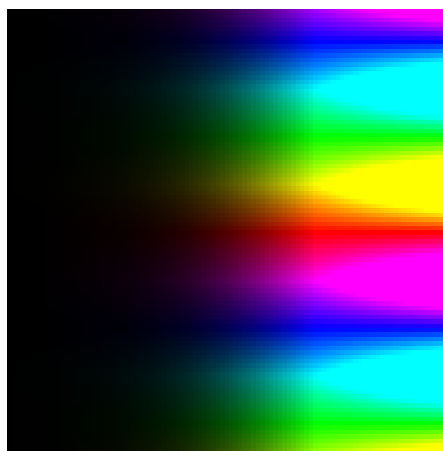
A very good example of these functions is addition, multiplication, and exponentiation. Addition is iterated succession, multiplication is repeated addition, and exponentiation is repeated multiplication. This chain goes on with the extension, as the following tetration is repeated exponentiation, and pentation is repeated tetration. Each of these functions is a superfunction of another. There are other hierarchies of such functions specifically “fast-growing hierarchy” but these are not currently required in this essay and will be included as a possible extension. The construction of superfunction may require different fixed points to be used for construction. According to the third requirement to ensure the conjugating of the input will conjugate the output, this condition is generally dismissed because tetration with complex bases does not meet the requirement.

## Preliminaries

This section will mostly consist of definitions and concepts that will be used. These concepts will mostly include complex analysis terminology.

### -Domain Colouring

Domain colouring is the primary visualization technique of Complex to Complex functions. For the function  $f(x+yi)=z+wi$  the coordinated  $(x,y)$  and the output in HSL (Hue-Saturation-Lightness) or a similar colour space, value (brightness/lightness)  $|z+wi|$  and output colour  $\arctan(z/w)$  or  $\arg(z+wi)$  are used to draw the image. The black points represent zeros and the white points represent poles (infinities). The color red is positive numbers and the colour cyan is negative numbers. Here is an example of a domain colouring depicting an exponential function ( $f(x)=e^x$ ). This method of graphing will be used extensively throughout this essay.



*Graph 1: Domain colouring of the exponential function  $e^x$*

### -Contour Integrals

Contour integrals are integrals in the complex domain that do not follow a straight line. Instead, these contours can be curved, have a loop and can even start where they end. For the most part, the contour integral is equivalent to the line integral. But if the function has poles or zeros this might not be the case. Contour integrals are shown as integrals with a  $\Omega$  or

$C$  (usage of  $C$  implies it is a line as  $\Omega$  is more general and is used for higher dimensions too) under them. And if a contour integral ending and starting points are the same, the integral will have a circle in it, these integrals are called Cauchy integrals. These integrals are defined by the Riemann sum (E. Jacobs) where  $z$  moves along the contour  $C$ .

Where:

$C$  is a contour in the complex domain.

$z$  is the complex variable of integration, moving along the contour  $C$ .

$\Delta z$  is the difference between consecutive points  $z_k$  and  $z_{k-1}$  on the contour  $C$ .

$f(z)$  is a complex function that we want to integrate along the contour  $C$ .

$$\int_C f(z) dz = \lim_{n \rightarrow \infty} \sum_{k=1}^n f(z_k) \Delta z$$

## Iterative Functional Equations

Iterative functional equations are a group of functional equations that encode self-similarity; these equations have two different appearances of the function that we are trying to find with different parameters. They all have a single parameter, which is the reason why they are sometimes called single-parameter functional equations. There are 18 (19 including superfunction being different than inverse Abel) iterative functional equations this essay will only make use of 4 of these equations:

$$\textit{Schröder} : \Psi(f(z)) = \Psi(z) \times s$$

$$\textit{Abel} : \alpha(f(z)) = \alpha(z) + 1$$

$$\textit{Inverse Schröder} : \Phi(z \times s) = f(\Phi(z))$$

$$\textit{Inverse Abel} : \sigma(z + 1) = f(\sigma(z))$$

Where throughout this essay  $\Psi$  will be used as the solution of Schröder's equation,  $\Phi$  will be used as inverse Schröder's equation,  $\alpha$  will be used as solution of the Abel's equation and  $\sigma$  will be used as the solution of inverse Abel's equation. Not forget  $\Psi^{-1}=\Phi$  and  $\alpha^{-1}=\sigma$ , these functions are inverses of each other, and this will be used later in functional analysis (E. Schröder, 1870) (N. H. Abel, 1826). Solutions of these equations are functions that satisfy the corresponding equation for all complex numbers  $z$ .

Eigenvalue  $s$  is the characteristic vector of the functional space on the fixed point  $L$  (vector space created by the function  $f$ ), in simple terms  $f(x)=sx$  when  $x$  is near a fixed point. The value of  $s$  in Schröder's equations are known as eigenvalues and can be determined by  $f'(L)$  but they are arbitrary and can be selected specifically but any selection other than  $f'(L)$  will yield deficient results generally in form  $F(x)=L$  or there can be no resulting function. The detailed explanation of eigenvalues in this context is beyond the scope of this essay.

The construction of superfunctions from Abel and inverse Abel is as follows:

$$\sigma(x + 1) = f(\sigma(x))$$

$$\sigma(x + n) = f^n(\sigma(x))$$

$$\textit{Substitute } x = \alpha(z)$$

$$\sigma(\alpha(z) + n) = f^n(z)$$

## -Fixed Points

Fixed points are such points for any function  $f$  that satisfies  $f(L)=L$ . This point can easily be found using Newton's root finding method and finding the root of  $f(L)-L=0$ . The fixed point is often shown by the variable  $L$ . There are a lot of classes of fixed points only 4 of them are in our area of concern, these fixed points consist of attracting, repelling, parabolic and super-attracting. (A. F. Beardon, 1991).

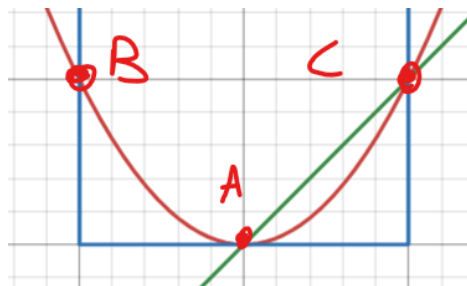
$$\textit{Attracting} : |f'(L)| < 1$$

$$\textit{Repelling} : |f'(L)| > 1$$

$$\textit{Parabolic} : |f'(L)| = 1$$

$$\textit{Super - Attracting} : |f'(L)| = 0$$

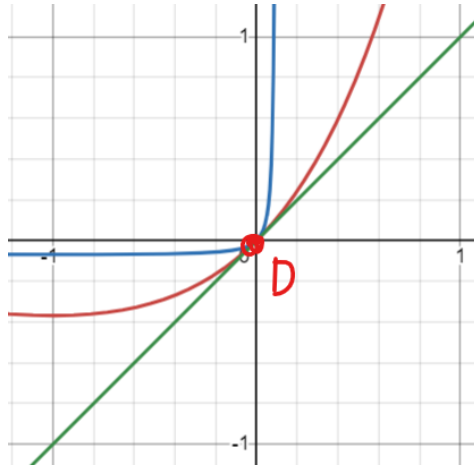
For the following graph, the green line is the identity  $y=x$  curve the red curve is the primary function  $f(x)$  and the blue curve is  $f^N(x)$  where  $N$  goes to infinity.



*Graph 2: Graph of a function with a super-attracting (A) fixed point a repelling (C) fixed point and a cyclic (B) fixed point. (The graphing is done using Desmos, additional annotation is done by me)*

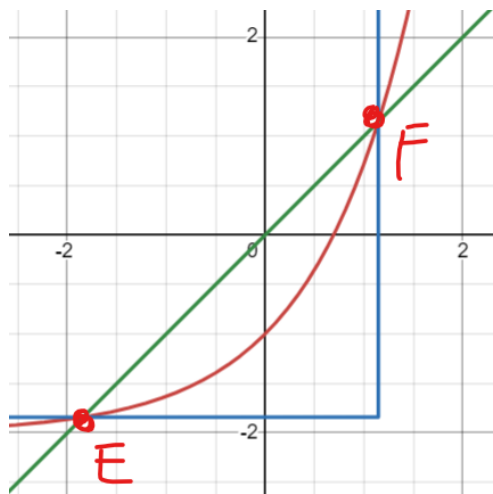
Where point A is a super-attracting fixed point, C is a repelling fixed point and B is a repelling-cyclic (different classification  $f(f(x))=f(x)$ ) fixed point. Super-attracting fixed points is problematic because a lot of the formulas include the  $f'(L)$  in their denominator, other than that they work similarly to attracting fixed points. Cyclic fixed points are out of the scope of this essay and will be included in the extension part.





*Graph 3: Graph of a function with a parabolic (D) fixed point.*

Point D is a parabolic fixed point, as can be seen after infinite iteration. This point will become the border of divergence and convergence, causing problems for both methods that rely on those properties.



*Graph 4: Graph of a function with an attracting fixed point (E) point and a repelling (F) fixed point.*

And point E is an attractive fixed point and point F is a repelling fixed point; both can serve the purpose and have similar formulas.

These points will be used to construct infinite compositions and will serve as points that do not diverge or points of convergence.

## Construction for the Solution of Schröder Equation

Let's assume he has a fixed point that is repelling  $L$ , we want to get such a function that  $\Psi(f(x)) = \Psi(x)s$ . We can iterate further by adding one more iteration. For convenience let  $s$  be the eigenvalue of the vector space created by the function and equal to  $f'(L)$  this thinking process of transforming complex functions into vector fields does help things simplify but in this essay, functional analysis will be prominent (M. Kuczma, 1968).

( $N$  will always used as the limit to infinity)

( $x, y, z$  are always used as free variables)

$$\Psi(f(x)) = \Psi(x)s$$

Substitute  $x = f(z)$

$$\Psi(f(f(z))) = \Psi(f(z))s$$

$$\Psi(f(f(z))) = \Psi(z)s^2$$

From here we can use induction

$$\frac{\Psi(f^{x+1}(z))}{\Psi(f^x(z))} = s$$

$$\Psi(f^x(z)) = \Psi(z)s^x$$

(this equation is sometimes called the 'zooming' equation)

Now we can construct the formula.  $f^N(x) = L$  if  $L$  is attracting and the converse  $f^{-N}(x) = L$  if  $L$  is repelling. So to find the  $\Psi$  we need to construct a function.

$f^{-N}(x)-L$  approaches 0 as  $N$  becomes larger.  $(f^{-N}(x)-L)s^N$  as  $N$  grows, if this expression converges then we can say that adding one more inverse of  $f$  is equivalent to dividing by  $s$ .

The reason why  $L$  is still here is that, as mentioned before, it is not possible to let everything grow by making everything revolve around a fixed point. As by induction:

$$\left(f^{-N}(x) - L\right)s^N = \left(f^{-(N+1)}(x) - L\right)s^{N+1}$$

$$\frac{f^{-N}(x) - L}{f^{-(N+1)}(x) - L} = s$$

$$\text{Let } g(x) = \left(f^{-(N+1)}(x) - L\right)s^N$$

$$g(f(x)) = f^{-N}(x) - L$$

$$\frac{g(f(x))}{g(x)} = s$$

$$\text{by definition } \frac{\Psi(f(x))}{\Psi(x)} = s$$

Thus, form equivalence:  $\Psi(x) = g(x)$

$$g(x) = \left(f^{-N}(x) - L\right)s^N$$

$$\Psi(x) = \left(f^{-N}(x) - L\right)s^N$$

The inverse  $\Phi(x)$  can be calculated by rearranging  $\Psi$

Start with the function:  $\Psi(x) = \left(f^{-N}(x) - L\right)s^N$

Let  $y = \Phi(x)$

Then:  $\Psi(y) = \left(f^{-N}(y) - L\right)s^N$

Substitute  $y = \Phi(x)$ :  $y = \left(f^{-N}(\Phi(x)) - L\right)s^N$

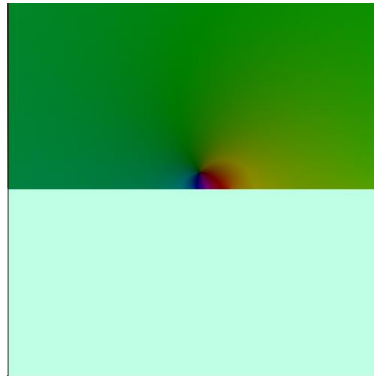
Divide by  $s^N$ :  $ys^{-N} = f^{-N}(\Phi(x)) - L$

Add  $L$ :  $ys^{-N} + L = f^{-N}(\Phi(x))$

Solve for  $\Phi(x)$ :  $\Phi(x) = f^N(L + xs^{-N})$

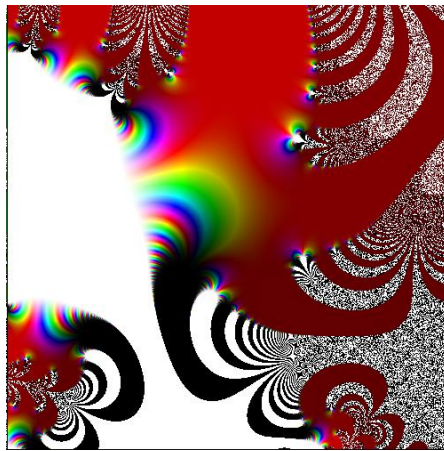
Conclusively;

$$\Psi(y) = \left(f^{-N}(y) - L\right)s^N$$



Graph 5: Schröder function of  $f(x)=e^x$ . (the lower plane grows too fast to be shown, fortunately using different fixed points for different planes solves this problem, (while evaluating the superfunction using  $L \approx 0.318132 + 1.33724i$  (6 s.f.) for the upper plane and  $L \approx 0.318132 - 1.33724i$  (6 s.f.) for the lower plane fixes the problem))

$$\Phi(x) = f^N(L + xs^{-N})$$



Graph 6: Inverse Schröder function of  $f(x)=e^x$ .

### -Transforming the Schröder Equation

From the Schröder function (solution of Schröder's equation) we can construct all 18 iterative functional equations (P. Walker, 1991). We mostly care about getting the Abel and inverse Abel functions. We have  $\Psi(f(x)) = \Psi(x)s$  and  $\Phi(xs) = f(\Phi(x))$  and need to construct:

$$\alpha(f(x)) = \alpha(x) + 1 \quad \text{and} \quad \sigma(x+1) = f(\sigma(x))$$

First, the normal (not inverse) variants:

$$\begin{aligned}\Psi(f^x(z)) &= \Psi(z)s^x \\ \log(\Psi(f^x(z))) &= \log(\Psi(z)s^x) \\ \log(\Psi(f^x(z))) &= \log(\Psi(z)) + \log(s^x)\end{aligned}$$

Let the log base be equal to s, so things simplify.

$$\log_s(\Psi(f^x(z))) = \log_s(\Psi(z)) + x$$

$$\text{Let } x = 1$$

$$\log_s(\Psi(f(z))) = \log_s(\Psi(z)) + 1$$

$$\alpha(f(z)) = \alpha(z) + 1$$

These two equations are equivalent thus:

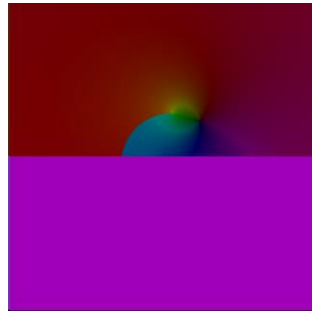
$$\alpha(z) = \log_s(\Psi(z))$$

And for the inverse Abel, we have an easier solution.

$$\begin{aligned}\alpha(z) &= \log_s(\Psi(z)) \\ \text{Substitute } z &= \sigma(w) \\ \alpha(\sigma(w)) &= \log_s(\Psi(\sigma(w))) \\ \alpha(\sigma(z)) &= z \text{ because } f(f^{-1}(x)) = x \\ w &= \log_s(\Psi(\sigma(w))) \\ s^w &= \Psi(\sigma(w)) \\ \Phi(s^w) &= \Phi(\Psi(\sigma(w))) \\ \Phi(\Psi(z)) &= z \text{ because } f^{-1}(f(x)) = x \\ \sigma(w) &= \Phi(s^w)\end{aligned}$$

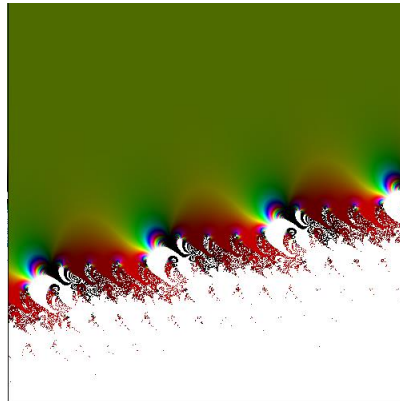
In open form:

$$\alpha(z) = \log_s(f^{-N}(z) - L) + N$$



*Graph 7: The Abel function of  $f(x)=e^x$  . (the lower plane grows too fast to be shown, again using different fixed points for different planes solves the problem)*

$$\sigma(z) = f^N(L + s^{z-N})$$



*Graph 8: Inverse Abel function of  $f(x)=e^x$ .*

## Construction of the Superfunction

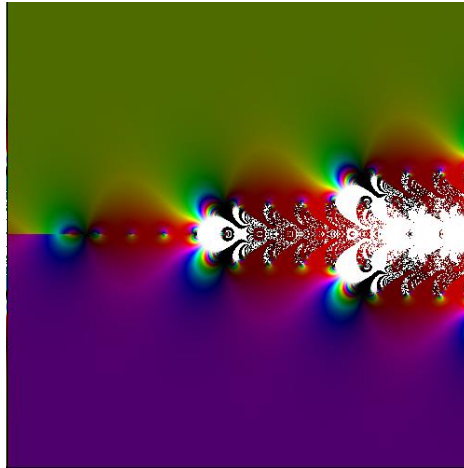
Now we have access to both Abel and inverse Abel we can easily construct the superfunction as follows

$$f^x(y) = \sigma(a(y) + x)$$

$$f^x(y) = f^N\left(L + s^{\log_s(f^{-N}(y) - L)} + x\right)$$

$$f^x(y) = f^N\left(L + (f^{-N}(y) - L)s^x\right)$$

$$f^x(y) = f^N\left(L + (f^{-N}(y) - L)f'(L)^x\right)$$

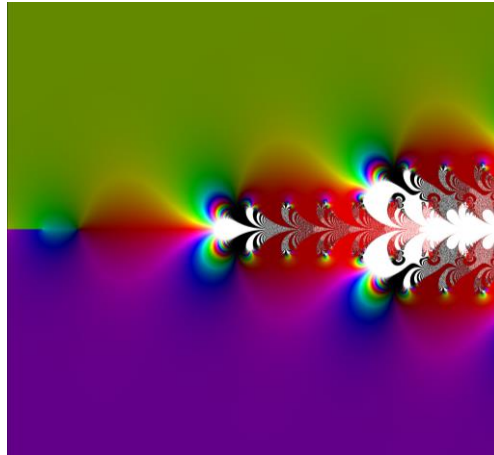


*Graph 9: A construction of tetration using the method that has been made does not satisfy real-to-real. (construction of tetration for real bases uses the conjugate of the fixed point for the construction of the lower plane)*

This construction of tetration is invalid because it does not satisfy the condition of being real to real. This is true for any function that has a fixed point not on the real line. Additional calculations, such as Riemann mapping, can be used to force these functions to be real on real. There exists a 1-cyclic function  $\theta(x+1)=\theta(x)$  such that  $F(\theta(z)+z)$  satisfies every condition of the superfunction equation, including being real on real. The calculation of the  $\theta$  function can be done in several ways, all of them complicated and out of the scope of this

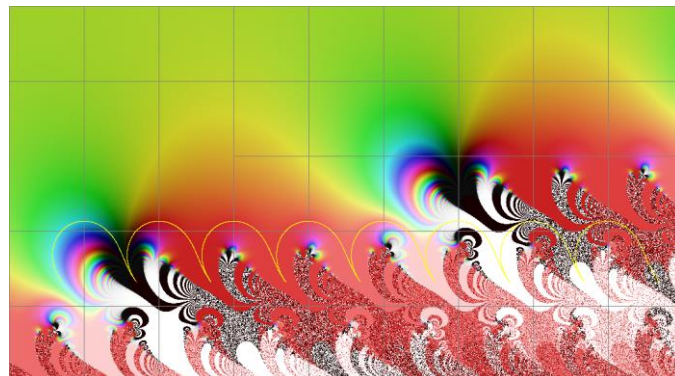


essay. But to provide light onto this subject here is the Kneser construction tetration (isomorphic to the formula we have constructed but with an additional Riemann mapping using a Fourier series).



*Graph 10: Kneser construction of tetration using Riemann mapping to construct a theta function to make tetration real to real.*

As can be seen, the addition of the theta function fixes the issue of being real to real. Although the calculation of the theta function is rather difficult and out of the scope of this essay. But in simple terms, Riemann mapping is a function that transforms the yellow line of  $\Im(F(z))=0$  ( $F(z)$  is real,  $\Im$  means the imaginary part) to the real line. Unfortunately, the method of calculating such a function is truly difficult.



*Graph 11: The yellow line represents where  $\Im(F(z))=0$  (L. Sheldon, 2017). This render does not belong to me.*

There exists an alternative method called the ‘sword track’ method or the equivalent Cauchy integral method. This method requires a contour integral to be calculated iteratively. However, simply it requires the following formula to be solved. In its most simplified way (D. Kouznetsov, 2009):

$$F(z) = \frac{1}{2\pi i} \oint_{\Omega} \frac{F(t) dt}{t - z},$$

Where  $\Omega$  is a circular integral around the point  $z$ .

The function  $F$  is the superfunction (tetration in this case).

$t$  is the variable of integration and  $z$  is the input parameter.

Using recursion, each iteration of this function will slowly converge to the true real-to-real tetration. Sadly, this method required taking iterated integrals, and thus very slow in comparison to other methods. However, this fixed method is accepted as the best alternative to Riemann mapping. This method allows up to 14 decimals to be calculated. The reason Riemann mapping is not preferred is that even in the original paper by Kneser, it has a lot of holes and the method is not fully given. These missing parts make that method very confusing in general. (The construction of these methods is out of scope for this essay)

### -Critical Points

The fixed point  $L$  may not behave well, there may not even be a fixed point present. First, the fixed point may live in the area of infinities for such functions  $f(\omega w + z) = 0$  where  $\omega w + z$  is a directional infinity, where  $\omega$  is the positive infinity,  $w$  is the direction and  $z$  is the offset. For these kinds of functions a  $\lim \rightarrow \infty$  might be required. And if there exists not

(linear functions) a fixed point then a higher order fixed point of  $f(f(L_2))=L_2$  or  $f(f(f(L_3)))=L_3$  can be used. If none exists, then there may not be a requirement for a fixed point to be fixed (shifting functions  $f(x)=x+z$ ) any complex  $L$  will produce a working solution. If not, this generalised method will unfortunately fail. An example of a function that fails to be created is  $f(x)=a^x$  where  $a$  is a real number smaller than  $\sqrt{2}$ . Other than this specific function, I was unable to find any function that failed. Fortunately, there exists another formula where the places of  $f^N$  and  $f^{-N}$  are switched. This method is commonly known as 'regular iteration' and is outside the scope of this essay.

Also when the fixed point  $L$  is repelling this construction will not work but there is an alternative solution of regular iteration which is rather well-known and will be elaborated further in this essay

This construction is flawed due to the possibility that  $f'(L)=0$  or  $f'(L)=1$  then the formula we have constructed will either produce the trivial solution of  $f(x)=L$  or the solution cannot be calculated at all. To prevent this an arbitrary variable  $c$  and a dependent variable  $\lambda$  will be added to the formula replacing  $f'(L)$ . This may seem problematic because  $s$  must be the eigenvalue of the function space. This statement is true, but consider that two divergent or deficient functions can come together to become convergent. The proof of convergence by replacing the eigenvalue with the arbitrary complex number  $c$  ( $c$  should not be equal to 0 or 1) is rather difficult. But intuitively it can be thought of as such;  $f^{-N}$  will cancel the growth of  $f^N$  but will leave an extra multiplicative term which we will deal with using the dependent variable  $\lambda$  as a coefficient. (The Abel and inverse Abel functions used with  $c$  are not Abel and inverse Abel functions but rather a variation of them)

First, let  $s = c$  and not  $s = f'(L)$

This will cause there to be some factor for both Abel and inverse Abel functions making them be.

$$\begin{aligned}\alpha(f(x)) &= \alpha(x) + \lambda \\ \sigma(x + \lambda) &= f(\sigma(x)) \\ \sigma(x + \lambda n) &= f^n(\sigma(x))\end{aligned}$$

Now we need a correction term lambda which can be calculated as such, the naive but powerful method is to do  $\lambda = \alpha(f(x)) - \alpha(x)$ .

Then the final construction will be:

$$\begin{aligned}f^x(y) &= f^N\left(L + \left(f^{-N}(y) - L\right)c^{\lambda x}\right) \\ f^x(y) &= f^N\left(L + \left(f^{-N}(y) - L\right)c^{x\left(\log_c\left(f^{1-N}(y) - L\right)\right) - \log_c\left(f^{-N}(y) - L\right)}\right)\end{aligned}$$

The formula might look big and scary, but there are many repeating parts, while coding on the computer, it can be greatly optimised and it takes very little space in the code (using Newton's method to find L).

## Construction of Generalized Abel Function

The generalised Abel or iterative functional equation is a functional equation in the form: given f and k we need to find F:

$$F(f(x)) = k(F(x))$$

Which generalizes over 16 of the 18 functional equations (Julia's equations can be constructed using this but it can't be given by only using this form as they require derivatives)

Let g be a function such that  $F(x) = g(\alpha(x))$

$\alpha$  is the solution of Abel's equation of f

$\sigma$  is the solution of the inverse of Abel's equation of  $f$

$K$  is the solution of the inverse Abel's equation of  $k$

$$k(F(x)) = F(f(x))$$

$$k(F(x)) = g(\alpha(f(x)))$$

$$k(g(\alpha(x))) = g(\alpha(f(x)))$$

$$k(g(\alpha(x))) = g(\alpha(x) + 1)$$

$$\text{Substitute } y = \sigma(x)$$

$$k(g(\alpha(y))) = g(\alpha(y) + 1)$$

$$k(g(x)) = g(x + 1)$$

$$g(x + 1) = k(g(x))$$

$$K(x + 1) = k(K(x))$$

$$K(x) \cong g(x)$$

(isomorphism here is used to indicate  $K$  and  $g$  may not be the same ( $g$  might not exist at all) but they both are inverse Abel of  $k$ , thus having similar enough characteristics that make this process work)

$$F(x) = g(\alpha(x))$$

$$F(x) = K(\alpha(x))$$

(here the definition of  $F$  is loose enough (we are trying to find a solution, not a unique one) that we can transform the isomorphism into equality)

From here we can form any function  $F$  that satisfies  $F(f(x))=k(F(x))$  if we have access to Abel's function and its inverse. Showing the strength of functional equations. From here we can solve any single-parameter functional equation.

## Extensions

Alternative hierarchies like Hardy's hierarchy and fast-growing hierarchy type superfunction go untouched in this essay as unlike classical superfunctions they are composed of  $f^x(x)$  type functions. In addition, it is generally more difficult to work with.

Julia's equation  $\beta(f(x)) = f'(x)\beta(x)$  and its inverses and conjugates cannot be formed using  $F(f(x))=k(F(x))$  unless a second argument for  $F$  is added, for example,  $F(x,y)$ , this goes unanalyzed in this essay. This methodology can also be used to create factorials and expofactorials ( $6^{5^{4^{3^2}}}$ ) (OEIS A049384) and similar concepts.

The Riemann mapping (the process of turning complex-to-complex functions into real-to-real functions) is untouched in this essay as it is complicated. On the other hand, Cauchy integrals are not generalisable and specifically built for constructing tetration; the addition of this content would fill the last missing hole in the construction of superfunctions, and further investigation is required in this area. This method is inconsistent in usage and requires specific actions depending on the function. Also requiring an integral to be calculated makes this method very slow.

Instead of this method of using Schröder's equation, there exists a method which utilizes matrices and Taylor series. This method is also untouched because the methodology is much different from what has been used here. Although this method will have a computation time in the range of hours instead of seconds. This method does not require fixed points, thus the problems caused by super-attracting and parabolic fixed points do not exist.

There exist many alternative ways to construct functions. But using Kneser's argument we can say the superfunction is unique. There exist many solutions to a functional equation. Unlike superfunctions, Abel, Schröder, and many other equations don't have 1 concrete result.

## Conclusions

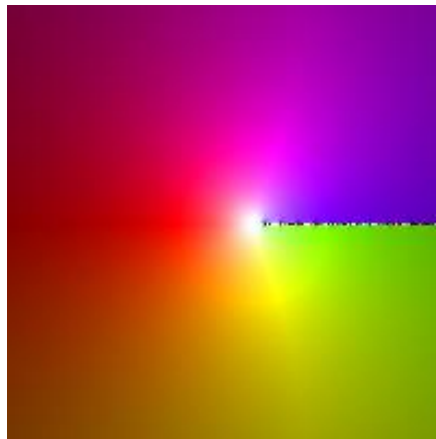
We have successfully generated a method for calculating superfunctions and even generalising Abel iterative functional equations further than it has ever done before. With the introduction of independent variable  $c$ , we have eliminated the critical points of parabolical fixed points and super-attracting fixed points and thus expanded the domain of the superfunction further.

The functions that have been generated will often have complex fractal-like features, and indeed they are fractals, specifically Fatou fractals. Fatou fractals, named after the French mathematician Pierre Fatou, also known as Julia fractals, are fractals that are generated as  $f^N(p)$  diverges or converges as  $N$  goes to infinity for each complex number  $p$ . The appearance of Fatou fractals in our results is not unexpected, given the inherent properties of the functions we are dealing with. The iterative nature of superfunctions, coupled with the complex dynamics involved in their construction, naturally gives rise to such fractal structures. In the formulas that are used to construct the superfunction and solutions to other functional equations, there are two infinite compositions which both could give rise to Fatou sets.

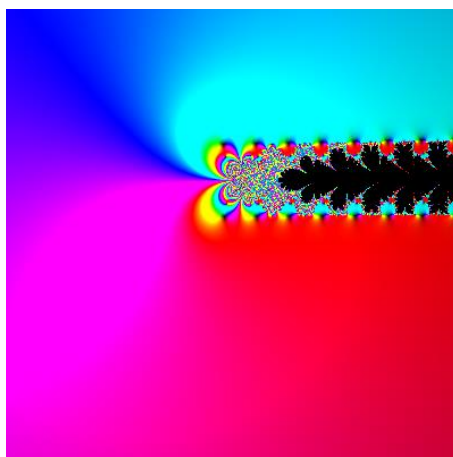
These functional equations have a place primarily in laser science. This new method of easier calculation of superfunctions can break some boundaries and eliminate some problems. In the area of googology, this new system enables us to use noninteger and even complex numbers for fast-growing functions such as tetration, pentation, or expofactorials.

## Gallery

Here are some examples of superfunctions that have been calculated using this method. The rendering engine is made by me and is created using the C++ and JavaScript coding language:

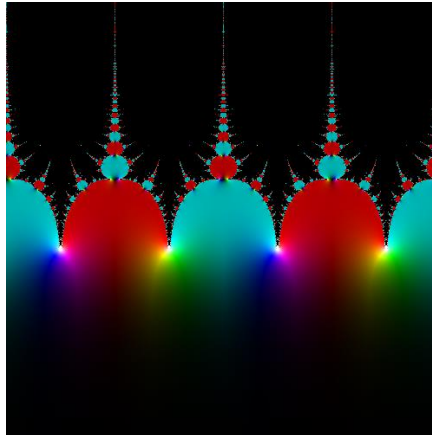


*Graph 12: supertan; superfunction of the tangent function  $\tan(x)$ , chaotic on the positive real line.*



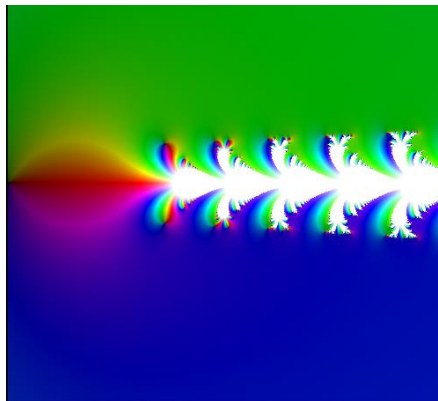
*Graph 13: supersin;  $F(0)=1$  superfunction of sine,  $\sin(x)$ , there are alternate periodic versions (Susin) also exist, but the construction we have used does not give that result.*



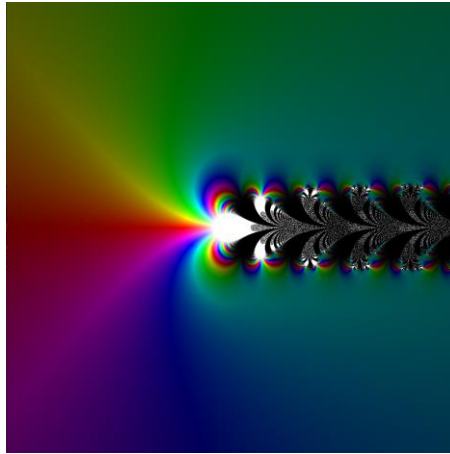


Graph 14: *Susin*, starting at  $F(0)=-1$ ; the inverse Abel of  $\sin$  that is real to real, can be constructed, inclusion of the Abel function in the definition will result same in graph 12 (D.

*Kouznetsov, 2014*). This is sometimes considered the ‘true’ superfunction of sine.



Graph 15: *supertrappman*; Trappman is the function  $\text{tra}(x)=x+e^x$  and this is the superfunction of it (H. Trappmann & D. Kouznetsov, 2011). This is important because there are no fixed points, but there exists a fixed point at negative infinity.



*Graph 16: superzex; zex is the function  $\text{zex}(x)=xe^x$  (H. Trappmann & D. Kouznetsov, 2011), the importance of this function is that it has a parabolic fixed point, the normal construction is unable to calculate it, and the additional variable  $c$  allows this to be calculated.*

As can be seen in these pictures, the structure of these functions is indeed very complex and fractal; from this, we can see that these functions are not easily computable or constructible without recursion or infinite iteration.

## Appendix

All the renders of domain colouring have been generated by either of the provided code. The only render that does not belong to me is the real-to-real tetration part, which has been rendered by Sheldon L.

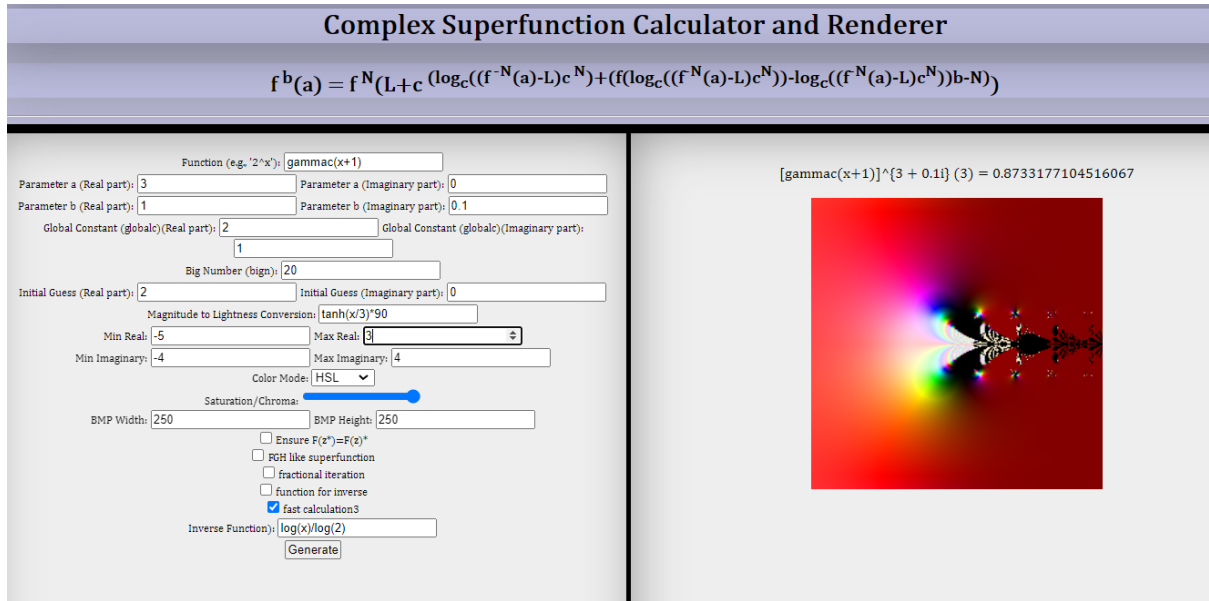


Figure 1: my online complex superfunction calculator and renderer website

(<https://tesertcode.github.io/CSC.html>) Many of the domain colourings in this essay are made with this tool that has been created by me.

```
function superfunc(funcStr, a, bf, globalc, bign) {
  //funcStr^bf(a)
  //bign = N -> lim infinity
  //globalc = c
  let bs = bf
  if (applyConjugate && bf.im < 0) {
    bs = math.conj(bs); // forcing f(conj(x))=conj(f(x))
  }
  try {
    let fi = parseComplex(bs);
    if (ffgh) {
      fi = parseComplex(bs); //fast growing hierarchy type superfunction
    } else {
      fi = parseComplex(a); //classical superfunction calculation
    }
    let lam = mathsc(funcStr, fi); //mathsc = f(x)
    for (let i = 0; i < bign; i++) { //infinite composition
      fi = findInverse(funcStr, fi, initialGuess, bign);
      lam = findInverse(funcStr, lam, initialGuess, bign); //calculating lambda
    }
    fi = math.divide(math.log(math.multiply(math.subtract(fi, fix), math.pow(globalc, bign))), math.log(globalc));
    lam = math.divide(math.log(math.multiply(math.subtract(lam, fix), math.pow(globalc, bign))), math.log(globalc));
    fi = math.add(fi, math.multiply(bs, math.subtract(lam, fi)));
    fi = math.add(fix, math.pow(globalc, math.subtract(fi, bign)));
    for (let i = 0; i < bign; i++) {
      fi = mathsc(funcStr, fi); //infinite composition
    }
    if (applyConjugate && bf.im < 0) {
      fi = math.conj(fi); // forcing f(conj(x))=conj(f(x))
    }
    scaleCanvas();
    return fi;
  } catch (error) {
    throw new Error('Calculation error: ' + error.message);
  }
}
```

Figure 2: the JavaScript code that is used for calculating superfunctions for the website.

(finds inverse and fix are calculated by Newton's root finding method)

```

function findFixedPoint(funcStr, initialGuess, tolerance = 1e-10, maxIterations = 100) {
  let x = parseComplex(initialGuess);
  const epsilon = 1e-10;
  for (let i = 0; i < maxIterations; i++) {
    const fx = math.subtract(mathsc(funcStr, x), x);
    const dfx = math.subtract(numericalDerivative(funcStr, x, epsilon), math.complex(1,0));
    const newX = math.subtract(x, math.divide(fx, dfx));
    if (math.abs(math.subtract(newX, x)) < tolerance) {
      return newX;
    }
    x = newX;
  }
  return initialGuess
}

```

Figure 3: the JavaScript code that evaluates fixed points that are used for my website.

```

function findInverse(funcStr, value, initialGuessa, bign) {
  const expri = math.compile(funcStr);
  const epsilon = 1 / bign;
  let x = 1;
  if(!fffx){ x = expri.evaluate({ x: value });}
  else{ x = parseComplex(initialGuessa);}
  const target = parseComplex(value);
  for (let i = 0; i < bign; i++) {
    const fx = mathsc(funcStr, x);
    const dfx = numericalDerivative(funcStr, x, epsilon);
    const newX = math.subtract(x, math.divide(math.subtract(fx, target), dfx));
    if (math.abs(math.subtract(newX, x)) < epsilon) {
      return newX;
    }
    x = newX;
  }
  return x;
}

```

Figure 4: The JavaScript code that is used to find the inverse of the function using Newton's method of root finding.

```

function numericalDerivative(funcStr, x, epsilon) {
  const fx1 = mathsc(funcStr, math.add(x, epsilon));
  const fx2 = mathsc(funcStr, x);
  return math.divide(math.subtract(fx1, fx2), epsilon);
}

```

Figure 5: The JavaScript code for taking derivatives with the limit definition used for my website.

```

double N=bign; //bign and N are  $\lim \rightarrow \infty$ 
cmpx fi=b; //cmpx is equivalent to std::complex<double>
if(cmpx(b).imag()<0.0) fi = conj(fi);
cmpx fia=fi//conj(x) is cmpx(x.real(),fabs(x.imag()))
cmpx bi=cmpx(fmod(todoub(fi),1.0),(tocomp(fi).imag()));
for(int i=0;i<24;i++){
fi+=knthetaa[i]*exp(cmpx(0.0,1.0)*bi*pi*2.0*(double)i);
} //knthetaa is the coefficients for the Riemann mapping
fi=fi+fia; //The values are precalculated
fi= cmpx(0.318132,1.33724)+pow(log(cmpx(0.318132,1.33724)),fi-N);
for(int i=0;i<N;i++) fi=exp(fi);
if(cmpx(b).imag()<0.0) fi = conj(fi);
return fi;

```

Figure 6: the C++ code that is used for Kneser's construction of tetration (`todoub()` is the real part of a complex number and `tocomp(x)` is the same as `cmpx(x,0)`).

## Bibliography

1. Kouznetsov, D. (2014). Evaluation of holomorphic ackermanns. *Applied and Computational Mathematics*, 3(6), 307-314.  
<https://doi.org/10.11648/j.acm.20140306.14>
2. Walker, P. (1991). Infinitely Differentiable Generalized Logarithmic and Exponential Functions. *Mathematics of Computation*, 57(196), 723–733.  
<https://doi.org/10.2307/2938713>
3. Kuczma, M. (1968). *Functional equations in a single variable*. PWN-Polish Scientific Publishers. <https://archive.org/details/functionalequati0000kucz>
4. Beardon, A. F. (1991). *Iteration of rational functions*. Springer.  
<https://doi.org/10.1007/978-1-4612-4422-6>
5. Kneser, H. (1949). Reelle analytische Lösungen der Gleichung und verwandter Funktionalgleichungen. *Journal für die reine und angewandte Mathematik*, 1949(187), 56–67.
6. Sheldon, L. (2017, May 18). Operational details (Implementation) of Kneser's method of fractional iteration of function  $\exp(x)$ . *Mathematics Stack Exchange*.  
<https://math.stackexchange.com/questions/2308409/operational-details-implementation-of-knesers-method-of-fractional-iteration>
7. Jacobs, E. (n.d.). *Introduction to contour integration*.  
<http://ftp.xecu.net/jacobs/vCalc/Complex/RiemannSumZ.pdf>
8. Kouznetsov, D. (2009). Solution of  $F(z+1)=\exp(F(z))$  in complex  $z$ -plane *Mathematics of Computation*, 78(267), 1647–1670. <https://doi.org/10.1090/S0025-5718-09-02188-7>
9. Schröder, E. (1870). Ueber iterirte Functionen. *Mathematische Annalen*, 3, 296–322.  
<https://doi.org/10.1007/BF01443922>
10. Abel, N. H. (1826). Untersuchung der Functionen zweier unabhängig veränderlichen Größen  $x$  und  $y$ , wie  $f(x, y)$ , welche die Eigenschaft haben, ... *Journal für die reine und angewandte Mathematik*, 1, 11–15.
11. Trappmann, H., & Kouznetsov, D. (2011). Computation of the two regular super-exponentials to base  $\exp(1/e)$ . *Mathematics of Computation*, 81(280).  
<https://doi.org/10.1090/S0025-5718-2012-02590-7>

12. Kouznetsov, D. (2014). Super sin. *Far East Journal of Mathematical Sciences (FJMS)*, 85(2), 219-238.